Лекция 4. Точечные коммуникации в MPI

29 сентября 2026 г.

MPI (Message Passing Interface) — API для реализации параллельных и распределенных вычислений с использованием модели распределенной памяти: каждый процессор имеет доступ только к своей области памяти, а пары «процессор — память» соединены друг с другом посредством сети.

Установка билиотеки в Ubuntu:

```
sudo apt install openmpi-bin openmpi-doc libopenmpi-dev
```

Простейшая MPI-программа показана в листинге ??.

```
#include <mpi.h>
#include <string.h>
#include <stdio.h>
#define MAX_SIZE 100

int main(){

        int comm_sz;
        int my_rank;
        char greeting[MAX_SIZE];

        MPI_Init(NULL, NULL);

        MPI_Comm_size(MPI_COMM_WORLD, &comm_sz);
        MPI_Comm_rank(MPI_COMM_WORLD, &my_rank);

        if (my_rank != 0){
                sprintf(greeting,
                        "Greetings from process %d of %d", my_rank, comm_sz);
                MPI_Send(greeting, strlen(greeting) + 1,
                        MPI_CHAR, 0, 0, MPI_COMM_WORLD);
        }
        else{
                printf("Greetings from process %d of %d\n",
                        my_rank, comm_sz);
```

```
                for (int i = 1; i < comm_sz; i++){
                        MPI_Recv(greeting, MAX_SIZE, MPI_CHAR,
                                i, 0, MPI_COMM_WORLD, MPI_STATUS_IGNORE);
                        printf("%s\n", greeting);
                }
        }

        MPI_Finalize();

        return 0;
}
```

Компиляция программы:

```
$ mpicc prog.c -o prog
```

Запуск параллельной программы (на одной машине на 4 процессорах):

```
$ mpiexec -n 4 ./prog
```

Запуск распределенного вычислительного процесса на двух машинках

```
$ mpiexec -hosts 2 ip_1 ip_2
```

Все команды `MPI` должны помещаться между вызовами функций `MPI_Init` и `MPI_Finalize`. Первая из них принимает указатели на аргументы командной строки и выделяет место для буферов передаваемых и получаемых сообщений, решает, какой ранг будет у какого процесса и т.д. Вторая функция освобождает выделенные ресурсы.

Коммуникатором называется множество процессов. Процессы могут отправлять и получать сообщения только в рамках одного коммуникатора. Коммуникатор `MPI_COMM_WORLD` состоит из всех процессов, которые запущены пользователем. В строках 14 и 15 происходит запись в переменные `comm_sz` и `my_rank` размера коммуникатора и порядкового номера текущего процесса в коммуникаторе соответственно.

Отправка и получение сообщений происходит путем вызова функций `MPI_Send` и `MPI_Recv`.

Сигнатура `MPI_Send`:

```
MPI_Send(void* buf,
        int size,
        MPI_Datatype,
        int dest,
        int tag,
        MPI_Comm communicator
);
```

Первые три аргумента описывают передаваемое сообщение: где оно хранится, сколько элементов какого типа содержит. Четвертый аргумент —

это номер процесса-получателя, пятый — тег, то есть условное обозначение, которое позволит получателю понять, какая информация содержится в этом сообщении. Последний аргумент — это имя коммуникатора.

Сигнатура `MPI_Recv`:

```
MPI_Recv(void* buf,
         int size,
         MPI_Datatype,
         int source,
         int tag,
         MPI_Comm communicator,
         MPI_Status* status_p
);
```

Первые три аргумента характеризуют получаемое сообщение: в какой буфер его записать, каковы максимальный размер и тип элементов сообщения. Далее указывается ранг процесса-отправителя и тег сообщения, а также имя коммуникатора. Последний аргумент предназначен для вывода информации о ранге отправителя и теге сообщения. Мы указали `MPI_STATUS_IGNORE`.

Пусть несколько процессов отправляют сообщения нулевому процессу. Может так оказаться, что, например, процесс с рангом 5 подготовил сообщение и готов его отправить, а процесс 1 все еще выполняет вычисления и не в состоянии осуществить отправку. Тогда, если процесс 0 ждет в функции `Recv` сообщения от первого процесса, то сеть будет простаивать, потому что процесс не может выйти из функции `Recv`, пока сообщение не получено. На помощь приходят т.н. `wildcard`-аргументы. В качестве третьего аргумента функции `MPI_Recv` можно передать макрос `MPI_ANY_SOURCE`, а в качестве четвертого — `MPI_ANY_TAG`. Можно указать один из этих тегов или оба. Тогда процесс сможет получить сообщение от того отправителя и/или с тем тегом, который в данный момент готов к получению.

Упомянутые выше `wildcard`-аргументы позволяют процессу не ждать какого-то конкретного сообщения, а получать то сообщение, которое отправитель может передать. Однако, если указан хотя бы один из этих макросов, то мы можем не знать тега сообщения или его отправителя, или и того, и другого. Также мы можем не знать размера сообщения. Тогда нужно создать переменную типа `MPI_Status` и передать указатель на нее в функцию `MPI_Recv`. Функция `MPI_Recv` запишет в эту переменную информацию о теге и отправителе, которую можно получить, обратившись к полям `MPI_SOURCE` и `MPI_TAG`. Узнать же размер передаваемого сообщения можно с использованием функции `MPI_Get_count`.

Что происходит, когда вызывается функция `MPI_Send`? В первую очередь передаваемая информация упаковывается в «конверт», который помимо содержательной части включает в себя служебные данные. Затем этот «конверт» может либо начать передаваться по сети получателю, когда тот вызовет соответствующую функцию `MPI_Recv`, либо происходит кэширование сообщения (сохранение во внутренней памяти) без его отправки. Обычно первый вариант развития событий происходит, когда размер передава-

емого значения выше некоторого порогового значения, а второй вариант характерен для маленьких сообщений. Таким образом, когда программа возвращается из функции `MPI_Send`, мы не знаем точно, состоялась ли коммуникация с процессом-получателем.

У функции `MPI_Send` есть блокирующий `MPI_Isend` и неблокирующий `MPI_Ssend` варианты.

Что же касается функции `MPI_Recv`, то возвращение из нее, напротив, говорит о том, что коммуникация состоялась.

Пусть процесс q вызывает функцию

```
MPI_Send(send_buf_p, send_buf_sz,
send_type, dest, send_tag, send_comm);
```

Процесс r осуществляет вызов

```
MPI_Recv(recv_buf_p, recv_buf_sz,
recv_type, source, recv_tag, recv_comm, &status);
```

Для того, чтобы случился акт коммуникации между процессами, нужно, чтобы

- `recv_comm=send_comm`
- `recv_tag=send_tag`
- `dest=r, source=q`

Для того, чтобы сообщение было передано успешно, нужно, чтобы дополнительно выполнялись еще условия

- `recv_type=send_type`
- `recv_buff_sz` $\geq$ `send_buff_sz`

Если процесс q посылает два сообщения процессу r, то первое посланное сообщение станет доступно на процессе r раньше второго. Если два разных процесса отправляют одному и тому же процессу сообщение, то порядок, в котором сообщения станут доступными получателю, зависит от производительности сети.

Взаимная блокировка (deadlock) — это ситуация, при которой несколько процессов находятся в состоянии ожидания ресурсов, занятых друг другом, и ни один из них не может продолжать свое выполнение.

Поставим задачу двум процессам поменяться своими значениями массива `a`:

```
1  if(my_rank == 0){
2    MPI_Send(&a, 10000, MPI_INT, 1, 0, MPI_COMM_WORLD);
3    MPI_Recv(&a, 10000, MPI_INT, 1, 0, MPI_COMM_WORLD, MPI_STATUS_IGNORE);
4  }
5  else if(my_rank == 0){
6    MPI_Send(&a, 10000, MPI_INT, 0, 0, MPI_COMM_WORLD);
```

```
7     MPI_Recv(&a, 10000, MPI_INT, 0, 0, MPI_COMM_WORLD, MPI_STATUS_IGNORE);
8 }
```

Данный код приведет к блокировке программы: процесс 0 будет заблокирован в строке 2, ожидая вызова первым процессом функции `Recv`. Однако по аналогичной причине и процесс 1 будет ожидать в строке 6 вызова функции `Recv` процессом 0. Отметим, что если сообщение достаточно короткое (например, передаем одиночные переменные вместо массивов), то блокировка с большой долей вероятности не произойдет, т.к. функция `Send` вместо блокировки процесса будет кэшировать сообщение и выходить из функции.

Решение проблемы взаимной блокировки состоит в использовании функции `MPI_Sendrecv`, которая состоит в одновременном вызове `MPI_Send` и `MPI_Recv`.

Сигнатура функции:

```
int MPI_Sendrecv(
        const void *sendbuf,
        int sendcount,
        MPI_Datatype sendtype,
        int dest,
        int sendtag,
        void *recvbuf,                  // OUT
        int recvcount,
        MPI_Datatype recvtype,
        int source,
        int recvtag,
        MPI_Comm comm,
        MPI_Status * status             // OUT
);
```

Если буфер отправки и буфер получения совпадают, то можно использовать следующий вариант:

```
int MPI_Sendrecv_replace(
        void *buf,                      // OUT
        int count,
        MPI_Datatype datatype,
        int dest,
        int sendtag,
        int source,
        int recvtag,
        MPI_Comm comm,
        MPI_Status *status              // OUT
);
```